

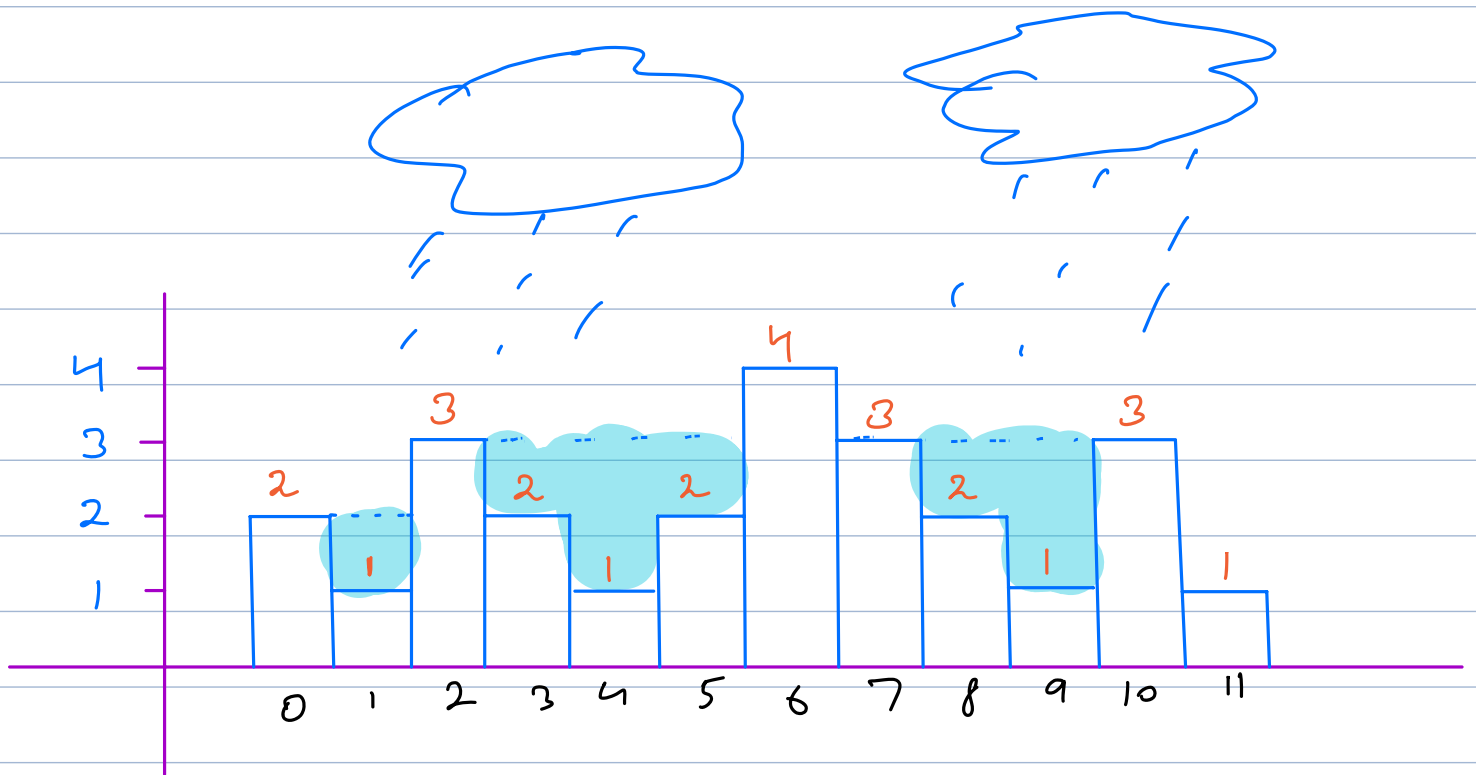
Starting 7:05

Today's Agenda:-

1. Trapping Rain Water
2. Spiral Matrix
3. Next Permutation

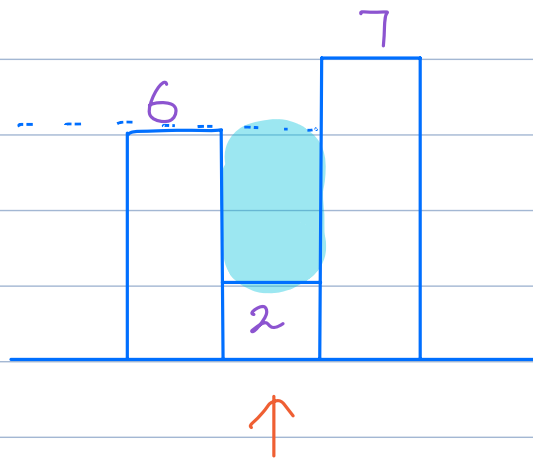
1. Given N buildings with height of each building. Calculate the amount of rain water trapped.

arr[] = {2, 1, 3, 2, 1, 2, 4, 3, 2, 1, 3, 1}

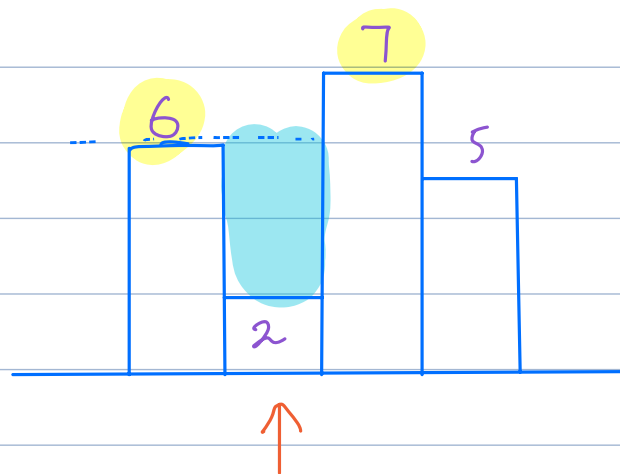


ans = 8 ;

Idea:- Try to evaluate the rain water that is trapped over each building..



ans = 4.



mm (6, 7)
= 6.

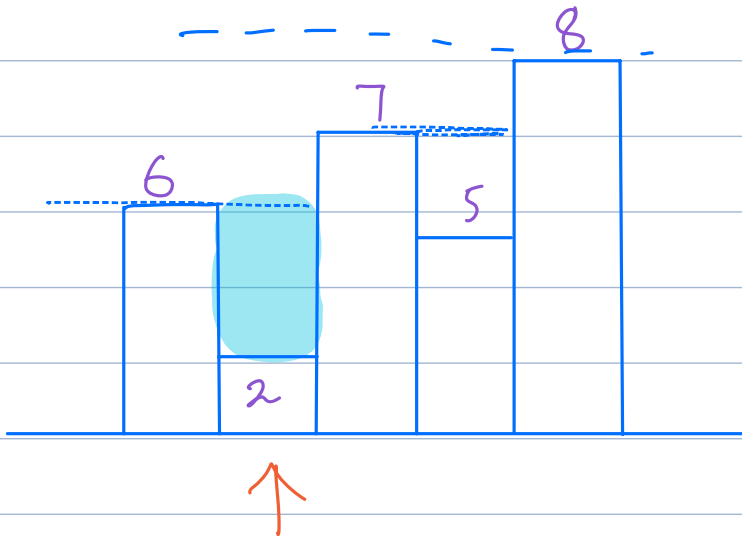
4,

$6 - h[i]$
 $= 6 - 2 = 4$.

$$\text{mn}(6, 8) = 8$$

$$6 - h + [i] = 4.$$

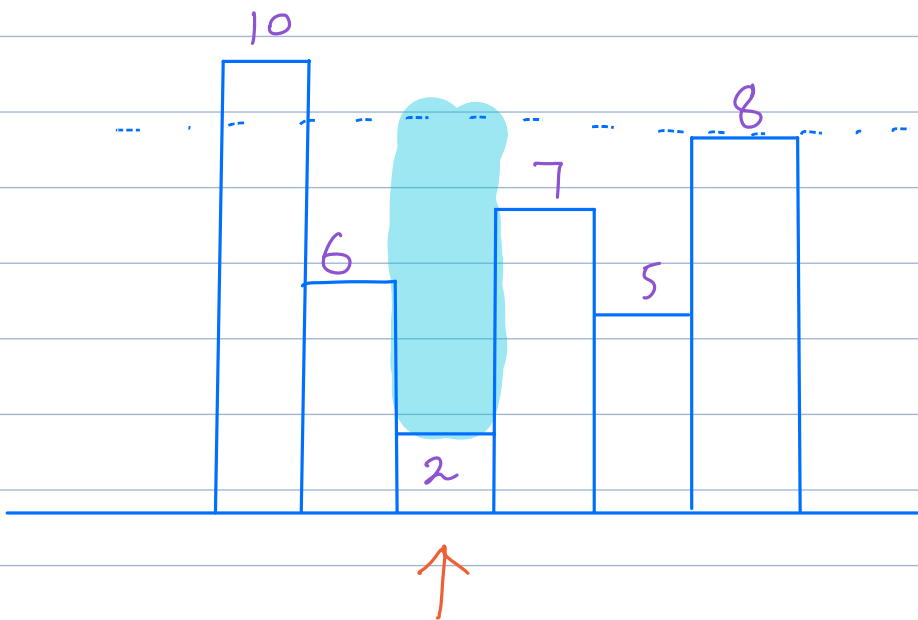
$$\underline{4.}$$

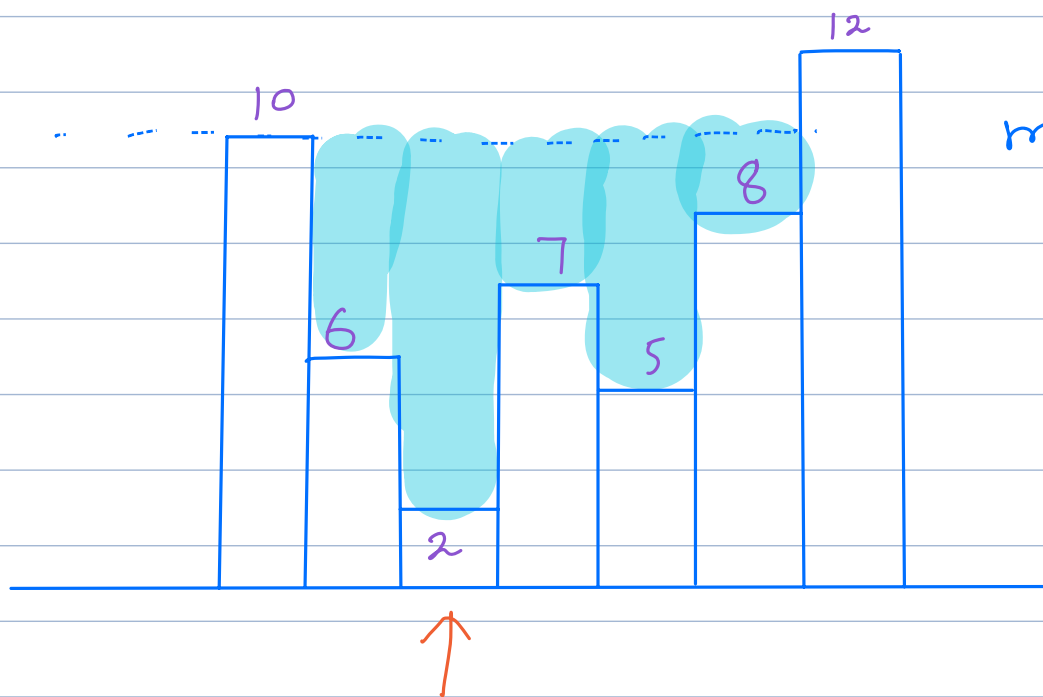


$$\text{mn}(8, 10) = 8$$

$$8 - 2 = \underline{6.}$$

$$\underline{6.}$$





$$\begin{aligned} \min(10, 12) \\ - 2 \\ = 10 - 2 \\ = 8 \end{aligned}$$

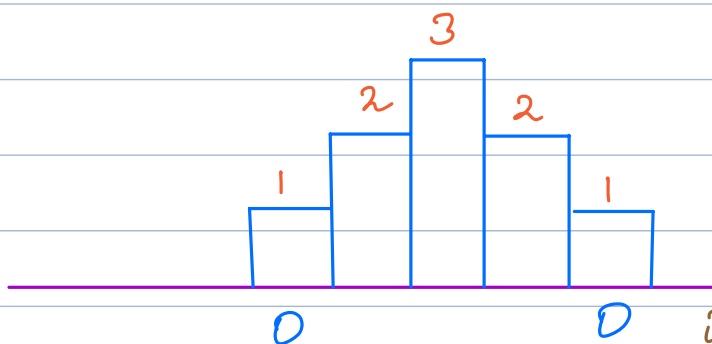
7:38

Conclusion The amount of water accumulated on top of i^{th} building is determined by $\max[0 \dots i-1]$ & $\max[i+1 \dots N-1]$;

So; level of water = $\min(\max[0, i-1], \max[i+1 \dots N-1])$

Amount of water = $\text{level} - \text{height}[i]$

Quiz 1



$$\text{ans} = 0$$

$$\begin{aligned} \text{level} &= \min(3, 1) \\ &= 1 \end{aligned}$$

$$1 - 2 = -1$$

Brute Force

Iterate over each building

for each building calculate $lmax$ & $rmax$.

$level = \min(lmax, rmax);$

$ans += \max(level - h[i], 0);$

Code

$[1, N-2]$.

$ans = 0;$

for ($i = 1; i < N-1; i++$) {

$N \left\{ \begin{array}{l} lmax = \max(0 \dots i-1); \\ rmax = \max(i+1 \dots N); \end{array} \right.$

$level = \min(lmax, rmax);$

$TC \rightarrow O(N^2)$

$ans += \max(level - h[i], 0);$

}

return ans;

Optimisation :-

We can store the lmax & rmax using **carry forward** approach.

$$\begin{aligned}\text{lmax}[i] &\rightarrow \max[0 \dots i]; \\ \text{rmax}[i] &\rightarrow \max[i \dots N-1];\end{aligned}$$

$$\text{lmax} = \{4, 4, 5, 7, 7, 7, 7, 7, 8, 8, 8\}$$

$$\text{arr} = \{ \overset{0}{4}, \overset{1}{2}, \overset{2}{5}, \overset{3}{7}, \overset{4}{4}, \overset{5}{2}, \overset{6}{3}, \overset{7}{6}, \overset{8}{8}, \overset{9}{2}, \overset{10}{3} \}$$

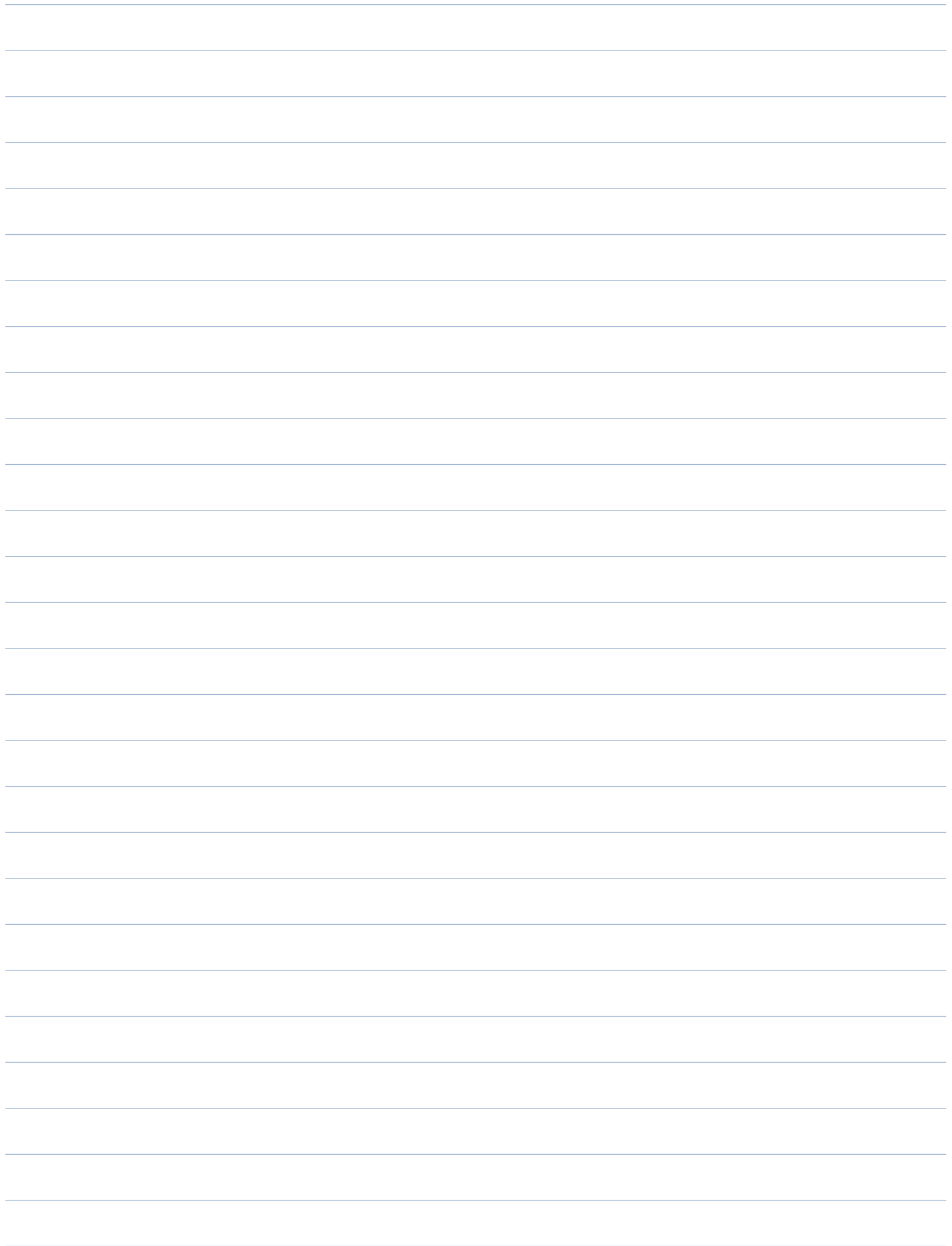
$$\text{rmax} = \{8, 8, 8, 8, 8, 8, 8, 8, 8, 3, 3\}$$

$$\text{lmax}[i] = \max(0 \dots i)$$

$$= \max(\text{lmax}[i-1], \text{arr}[i]);$$

$$= \max(\text{lmax}[i-1], \text{arr}[i]);$$

$$\begin{aligned}\text{rmax}[i] &= \max(i, N-1); \\ &= \max(\text{arr}[i], \text{rmax}[i+1]); \\ &= \max(\text{arr}[i], \text{rmax}[i+1]);\end{aligned}$$



Code

1. Prepopulate the $lmax$ & $rmax$ arrays using carryforward technique.

N { $lmax[N];$
 $lmax[0] = a[0];$
for ($i=1; i < N; i++$) {
 $lmax[i] = \max(lmax[i-1], a[i]);$
}

N { $rmax[N];$
 $rmax[N-1] = a[N-1];$
for ($i=N-2; i \geq 0; i--$) {
 $rmax[i] = \max(a[i], rmax[i+1]);$
}

2. Use $lmax$ & $rmax$ arrays to calculate the level in a single iteration.

ans = 0;

for (i = 1; i < N-1; i++) {

maxL = lmax[i-1];

maxR = rmax[i+1];

level = min(maxL, maxR);

ans += min(level - ht[i], 0);

}

return ans;

T.C $\rightarrow O(N)$

S.C $\rightarrow O(N)$

Problem 2:- Given a Square Matrix. Print its boundary elements. (clockwise directions)

mat[N][N]

	0	1	2	3	4
0	1	2	3	4	5
1	6	7	8	9	10
2	11	12	13	14	15
3	16	17	18	19	20
4	21	22	23	24	25

mat[5][5]

o/p:- 1, 2, 3, 4, 5, 10, 15, 20, 25, 24, 23, 22, 21, 16, 11, 6

Q:- 2

1	2	3
4	5	6
7	8	9

o/p:- 1, 2, 3, 6, 9, 8, 7, 4

Approach:-

mat[N][N]

mat[5][5]

	0	1	2	3	4
0	1	2	3	4	5
1	6	7	8	9	10
2	11	12	13	14	15
3	16	17	18	19	20
4	21	22	23	24	25

Print first N-1 elements

- ① 0th row →
- ② N-1th column ↓
- ③ N-1th row ←
- ④ 0th col ↑

Code

i = 0; j = 0;

[1, N)
↓
N-1

First row

```
for (t=1; t<N; t++) {  
    | print (arr[i][j]); j++;  
    |  
    | 3
```

T.C → O(N)

Last Column

```
for (t=1; t<N; t++) {  
    | print (arr[i][j]); i++;  
    |  
    | 3
```

Last Row

```
for (t=1 ; t<N; t++) {  
    print (arr[i][j]); j--;  
}
```

First Column

```
for (t=1 ; t<N; t++) {  
    print (arr[i][j]); i--;  
}
```

Problem 3:- Spiral Printing of a square matrix.

mat[N][N]

	0	1	2	3	4	5
0	1	2	3	4	5	6
1	7	8	9	10	11	12
2	13	14	15	16	17	18
3	19	20	21	22	23	24
4	25	26	27	28	29	30
5	31	32	33	34	35	36

6, 6

0, 0

4, 4

(1, 1)

2, 2

(2, 2)..

o/p → [1, 2, 3, 4, 5, 6, 12, 18, 24, 30, 36, 35, 34, 33, 32, 31, 25, 19,

13, 7, 8, 9, 10, 11, 17, 23, 29, 28, 27, 26, 20, 14, 15, 16, 22, 21]

Q:- 3.

13	14	12	8
9	1	2	7
0	4	3	0
10	5	6	11

o/p:- { 13, 14, 12, 8, 7, 0, 11, 6, 5, 10, 0, 9,
1, 2, 3, 4 }

Approach

Print the boundary elements of the matrix $N \times N$. with s.p. (90)
Once done, reduce $N = N - 2$. &
Update $i = i + 1$ & $j = j + 1$ to move to the inner boundary.
Repeat until $N \leq 0$.

$i=0; j=0; N$

while ($N \geq 1$) {

First row

for ($t=1; t < N; t++$) {
1 print ($arr[i][j]$); $j++$;
3

Last Column

for ($t=1; t < N; t++$) {
1 print ($arr[i][j]$); $i++$;
3

Last Row

for ($t=1; t < N; t++$) {
1 print ($arr[i][j]$); $j--$;
3

First Column

for ($t=1; t < N; t++$) {
1 print ($arr[i][j]$); $i--$;
3

$N -= 2; i++; j++;$

3

if ($N == 1$) {
1 print ($arr[i][j]$);
3

	0	1	2	3	4	5	6
0	1	2	3	4	5	6	7
1	8	10	12	17	18	19	20
2	9	11	13	21	22	25	26
3	14	15	16	23	24	27	28
4	29	31	35	36	37	38	39
5	30	32	40	41	42	43	44
6	33	34	45	46	47	48	49

$N = \chi$; $\hat{\tau} = \emptyset$; $\hat{\eta} = \emptyset$
~~8~~ ~~1~~ ~~1~~
~~3~~ ~~2~~ ~~2~~
 1 3 3

8:50



Next Permutation

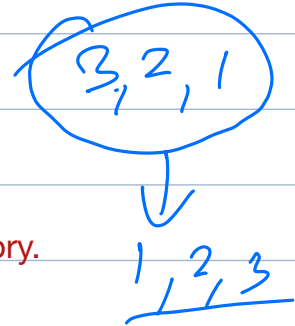
< Question > : Implement the next permutation, which rearranges numbers into the numerically next greater permutation of numbers for a given array A of size N.

If such arrangement is not possible, it must be rearranged as the lowest possible order, i.e., sorted in ascending order.

****NOTE:****

The replacement must be in-place, do not allocate extra memory.

DO NOT USE LIBRARY FUNCTION



ex:- { 1, 2, 3 }

↓

{ 1, 3, 2 }

All Perms:-

1 2 3
1 3 2
2 3 1
2 1 3
3 1 2
3 2 1

ex:- { 3, 2, 4 }

↓

{ 3, 4, 2 }

All Perms

2, 3, 4
2, 4, 3
3, 2, 4
3, 4, 2
4, 2, 3
4, 3, 2

4, 3, 2

5, 6

6, 5

7, 6, 5

5, 6, 7

5, 7, 6

6, 5, 7

6, 7, 5

7, 5, 6

7, 6, 5

a > b > c > d > e.

1, 9, 8, 3, ~~4~~, 7, 6, ~~5~~

↓
same

1, 9, 8, 3, 5, 4, 6, 7

exr { 1, 2, ~~3~~, 6, 5, ~~4~~, 1, 3 }.

↓ same.

{ 1, 2, 4, 1, 3, 5, 6 }

ex:-

2 3, 6, 5, 9, ~~2~~, 7, ~~8~~, 3

6

2



same

{ 3, 6, 5, 9, 6, 2, 7, 3 }

Steps:

N { ① Start iterating from R.H.S & see if $arr[i] < arr[i+1]$;

② Iterate from $N-1$ to i & see if $arr[idx] > arr[i]$; if yes, break immediately.

③ Swap ($arr[i]$, $arr[idx]$);

④ Reverse the array from idx ($i+1$ to $N-1$);

T.C $\rightarrow O(N)$

Code $\rightarrow$ H.W.



